Deterministic specialization for iPRA

Removing the term argument from clause application

Prepared for Vladimir Reshetnikov

Abstract

The current ConciseIPRA proposals already reduce full intuitionistic primitive recursive arithmetic to five rule patterns on equations and one paired binder. We make a further inferential restriction: a clause application supplies no numerical target. The newest clause is always specialized at one, and every older clause at the newest eigenparameter. All arbitrary numerical specialization remains derivable.

The completeness proof names the intended term by an existential equality. Its main difficulty is structural: extending a scope changes the permitted target of an existing inference. A rank-recursive transport construction and a proof-height compiler regenerate premises under the larger scope; unrestricted weakening is not assumed. We prove exact consequence equivalence with the existing clause calculus, preserving its whole-domain bi-interpretation and equiconsistency with full first-order iPRA. Both numerical interpretation composites have graph $x = y$.

The complete readable checker has 2914 normalized bytes, 819 significant tokens, and 896 AST nodes, versus 2950, 829, and 902 for the incoming uniform-Step checker. We separate the small rule-interface reduction from an independent elimination of redundant scope state. A separate implementation of the unchanged native Eq/Q/D calculus reduces its fully validated checker from 866 to 820 bytes. These are controlled improvements, not a proof of an absolute minimum. The package includes complete written arguments, a bidirectional executable translation, 100 forward and reverse certificate checks, and independent boundary audits. It does not claim a new Lean or Rocq formalization.

Frozen source. ProveIt, commit
`5d85023b0688e03b75f0ec3a04bb43ef0bb51705`.
Main change. An arbitrary-term inference becomes deterministic.
Tradeoff. Smaller checking interface; potentially much larger proofs.

Contents

1 Result, baseline, and precise claim **2**
1.1 What is counted and what is inherited . . . 2

2 The complete deterministic-specialization calculus **3**
2.1 Syntax, computation, and all primitive rules . . . 3
2.2 The arithmetic transport used in the proof . . . 4
2.3 Exact consequence equivalence . . . 4
2.4 Ordinary realization and the interpretation composites . . . 6

3 Closing the bridge to the inherited axiomatization **7**
3.1 Normalization and the arithmetic interface . . . 7
3.2 All case introductions and all equation inductions . . . 7
3.3 Why this is full intuitionistic first-order arithmetic . . . 8

4 The complete checker and certificate contract **9**
4.1 Concrete syntax and scopes . . . 9
4.2 Eliminating redundant scope state . . . 10
4.3 The executable proof transformation . . . 10

5 Controlled size measurements and verification **10**
5.1 Verification evidence . . . 11

6 An independent 820-byte native checker refinement **12**

7 Reproduction, provenance, and limits **12**

A Inherited mathematical sources **13**

B Complementary introductions with one local assumption **13**

C Arithmetic strength and both ordinary interpretation composites **19**

D Internal scalar predecessor bootstrap **20**

E From literal-equation induction to full intuitionistic PRA **25**

F Pairing, primitive recursion, and internal numerical composites **30**

G Complete primary checker **33**

1 Result, baseline, and precise claim

The question asks for a presentation simpler than the current `ConciseIPRA` proposal, while preserving full iPRA up to bi-interpretation. We use the complete checker as the implementation unit and include all syntax validation, scopes, normalization, inference checking, JSON input, and command-line output. The comparison is pinned to the commit on the title page, not to a moving branch [1].

The repository index describes fifty-six filed research packages. Its five incoming archives were also inspected: computation normal form, counter evaluation, reduced clause calculus, root-free clausal logic, and uniform successor Step. In particular, the five-rule architecture, zero-normal iterators, Refl plus uniform Step, and the clause interface were already available [2, 3, 4]. Combining those ingredients alone would not be a new mathematical reduction.

The present change concerns the remaining arbitrary specialization argument. An unrestricted clause instruction has the shape

$$[\mathtt{u},\ i,\ t,\ \pi],$$

where i chooses a retained clause, t is any numerical term, and π proves its antecedent. Our instruction is

$$\boxed{[\mathtt{u},\ i,\ \pi]}.$$

The permitted target is computed from the clause position and the current parameter scope. No hidden arbitrary term, mode bit, substitution certificate, or evaluator is moved to another field of this instruction. The existing existential introduction still supplies a witness; the completeness translation uses that existing facility to name the desired term.

Theorem 1.1 (Main result, stated in advance)**.** *The deterministic five-rule calculus of Section 2 has exactly the same consequence relation as the unrestricted zero-normal clause calculus. Its ordinary realization is bi-interpretable and equiconsistent with full first-order intuitionistic primitive recursive arithmetic. Both interpretations use the whole numerical domain and actual equality; the internally definable isomorphisms for the numerical composites are $x = y$.*

The mathematical improvement is a restriction of a primitive rule, with an effective recovery of every omitted instance. The rule count stays five. It is not a deletion of induction, equality, or a logical connective. There is also no theorem here that the resulting presentation is globally simplest among all effective presentations. Such a statement would require a fixed representation and a much stronger lower-bound argument than a matched implementation comparison.

1.1 What is counted and what is inherited

The selected numerical language, paired connective, iterator arithmetic, and scalar-to-iPRA bridge are inherited. We reproduce their essential arguments and the five authenticated bridge sources in the appendices. The new contribution is deterministic specialization, its persistence proof, and the executable compiler. The separate grammar/scope traversal fusion is an implementation refinement of the old native calculus.

These contributions should not be conflated. The 2914-byte readable checker implements the new deterministic clause calculus. The 820-byte supplement implements the unchanged, differently encoded native Eq/Q/D calculus. It is not a 820-byte implementation of the new calculus. All measured programs import only the Python standard library, and their complete sources are supplied.

2 The complete deterministic-specialization calculus

2.1 Syntax, computation, and all primitive rules

There is one numerical sort. Start from finite raw terms

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u in h is bound by the iterator; n and a are outside that binder. The zero-normalization map acts recursively on subterms and contracts $I(0, a; u.h)$ to a, but does not contract successor-counter iterators. The terms of the present calculus are the fixed points of this map. Every substitution is capture avoiding and followed by this normalization, including a substitution into an iterator counter that newly becomes zero. For readability, expressions below always denote their zero-normal forms.

Formulas and clauses have grammars

$$F ::= (s = t) \mid D_{x,y}F, \qquad [A; y.B(y)] \quad (y \notin \mathrm{FV}(A)).$$

Both x and y are bound by D. A context is a finite ordered list of clause occurrences; occurrences remain distinct even when their contents agree. An ordinary assumption A is represented by the constant clause $[0 = 0; y.A]$, with y unused. Put $1 = S0$, and designate $0 = 1$ as contradiction. Clause order is used only to choose a specialization target; clauses are reusable, and no assumption is consumed by a rule.

The following five schemas are the entire primitive inventory. There are no additional logical, equality, computation, successor, induction, weakening, exchange, or cut rules.

$$\frac{}{\Gamma \vdash t = t}\ (\mathsf{Refl}), \qquad \frac{\Gamma \vdash I(c, a; u.h) = t \qquad \Gamma \vdash I(Sc, a; u.h) = r}{\Gamma \vdash r = h[t/u]}\ (\mathsf{Step}).$$

In Step the first iterator is normalized, so it is just a when c is zero. Repeated expressions must be syntactically identical; no hidden equality decision or evaluation oracle occurs in this rule.

$$\frac{\Gamma \vdash D_{x,y}B(x, y) \qquad [B(b, 0); y.B(b, y)], \Gamma \vdash F}{\Gamma \vdash F}\ (\mathsf{DE}).$$

Here b is fresh for Γ, F and all passive matrix parameters.

$$\frac{[0 = 0; v.B(t, \eta)], \Gamma \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x, y)}\ (\mathsf{DI}), \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \text{ is literally } 0, \\ 0, & \text{otherwise.} \end{cases}$$

Here v is unused, and z is fresh for Γ, t and all passive matrix parameters. An equation beneath a D is not a literal equation. The fifth schema is Use_* below. The witness-selected DI convention and its recovery of the missing equation-case instances at witness zero are inherited from the existing uniform-Step calculus; they are not altered by the present specialization reduction.

Store context clauses newest first. For a nonempty parameter scope let b_* be its last parameter; for the empty scope put $b_* = 0$. Replace unrestricted clause use by the following rule:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(v_i)} \quad [A; y.B(y)] = \Gamma_i, \qquad v_i = \begin{cases} 1 & i = 0, \\ b_* & i > 0. \end{cases} \qquad (\mathrm{Use}_*)$$

Thus the newest clause is used at $1 = S0$; every older clause is used at the current last parameter. The certificate supplies its clause index and antecedent proof, with no numerical target or mode bit.

2.2 The arithmetic transport used in the proof

For $J_s = I(1, s; u.u)$, Step sends premises $s = t$ and $J_s = r$ to $r = t$. If $p : s = t$ and $q : s = r$, then

$$\mathsf{Step}(p, \mathsf{Step}(q, \mathsf{Refl}(J_s))) : r = t. \tag{Pivot}$$

This gives symmetry by putting the first premise equal to $s = s$, and then transitivity. For a term template h, put $K = I(1, s; u.h)$. Step on reflexivities gives $K = h(s)$; Step on $p : s = t$ and that equation gives $h(s) = h(t)$. Hence transport in an equation template $F(v) = (a(v) = b(v))$ follows from

$$a(t) = a(s), \qquad a(s) = b(s), \qquad b(s) = b(t).$$

This derivation uses only Refl and Step and opens no new scope. Applying Step to the two appropriate reflexivities also gives successor computation. In particular $C(0, a, b) = a$ is definitional and $C(Sz, a, b) = b$ is derivable for the selector $C(y, a, b) = I(y, a; u.b)$. These observations prove every arithmetic operation needed by the specialization construction independently of the logical rules.

2.3 Exact consequence equivalence

Theorem 2.1 (Deterministic-specialization equivalence)**.** *The resulting calculus derives exactly the same open sequents as the unrestricted five-rule clause calculus. The ordinary numerical and formula interpretations are unchanged.*

Proof. Soundness is immediate because every new use is an old use. We give a constructive completeness proof. Scope extension is not assumed: it changes the last parameter, and inserting a clause changes which clause is newest. Those two effects are handled by the proof-producing procedures below.

Let $C(y, a, b) = I(y, a; u.b)$, and let

$$N_t = D_{x,y}(C(y, 0, x) = C(y, 0, t)). \tag{Name}$$

Introduce N_t at witness t. Its required successor equation has literally equal sides, so reflexivity proves the branch in either introduction mode. Eliminate N_t and call the fresh parameter b. The newly inserted clause is

$$\mathcal{N}_{b,t} = [0 = 0;\ y.C(y, 0, b) = C(y, 0, t)]. \tag{N}$$

Since it is newest, Use_* applies it at 1. Reflexivity supplies its antecedent, and selector computation gives $b = t$. This particular proof is valid immediately after insertion; we do not pretend that it automatically remains valid after further context extensions.

A *persistent premise procedure* is a procedure that produces a proof of its conclusion after every injective renaming of its free parameters and every embedding of its retained clause occurrences into a larger context and scope. This definition describes an external finite proof-producing algorithm, not an added structural inference rule. A retained constant clause for an ordinary assumption is persistent: either permitted target gives the same formula. Refl and arithmetic combinations of persistent premises are also persistent.

Atomic specialization comes first. Let $[A; y.E(y)]$ be any retained clause whose body E is a literal equation, let P be a persistent procedure for A, and let t be any term. To specialize E at t, introduce and eliminate N_t. In its branch, request P in the enlarged context, and apply the original clause at the new last parameter b. The original clause is now older, so this is exactly its permitted use. The result is $E(b)$. The immediately available proof $b = t$ from (N), together with equational congruence, gives $E(t)$. Close the elimination. Call this procedure U_0. The arithmetic transport here opens no scope, so it does not require a persistent proof of $b = t$. The whole recipe is uniform under context and scope embeddings, so U_0 produces a persistent output whenever P is persistent.

In particular, after (N) has ceased to be newest, its equation $b = t$ still has a persistent proof: use U_0 to specialize that retained atomic clause at 1, with Refl of zero as its persistent antecedent, and apply selector computation. This use of U_0 is not circular. Its own auxiliary naming clause is newest when used at 1, and its final transport is atomic. This is the essential extra step that permits the deterministic protocol.

Mutual recursion by formula rank. Rank is the number of D binders above the final equation. We construct T_r and U_r for every finite rank r:

- T_r transports a formula F of rank r along a persistent proof of $s = t$, from an arbitrary supplied proof of $F(s)$ to a proof of $F(t)$. The supplied proof of $F(s)$ need not be persistent.
- U_r specializes a retained clause $[A; y.F(y)]$, with F of rank r, at an arbitrary term t, provided its antecedent A is supplied by a persistent procedure. It produces a persistent proof procedure.

For T_0, use the equational congruence already derived from Step; U_0 was constructed above.

Assume the operations are known below rank r, and write $F(v) = D_{x,y}G(v, x, y)$. For T_r, use the arbitrary supplied proof of $F(s)$ immediately as the major premise of DE. Let its fresh witness be c. The context now contains $[G(s, c, 0); y.G(s, c, y)]$. Introduce $F(t)$ at witness c. Because c is a fresh parameter, it is not literally zero; this is a case introduction, including when G is an equation. The local assumption $G(t, c, 0)$ is a constant clause and therefore persistent. Transport it back along the persistent equality by T_{r-1}, obtaining a persistent proof of $G(s, c, 0)$. Use U_{r-1} to specialize the retained closure at Sz, then transport forward by T_{r-1} to obtain $G(t, c, Sz)$. Close DI and DE. The original arbitrary minor proof was used outside the new scope and was never weakened. The recipe is persistent when that minor premise is supplied persistently.

Now construct U_r. Introduce and eliminate N_t, obtaining the newest parameter b. Request the antecedent procedure in that extension and use the original, now older, clause at b. This yields $F(b)$. The equality $b = t$ has the persistent proof from the atomic-specialization paragraph, even under the scopes later opened by T_r. Apply T_r to obtain $F(t)$, and close the naming elimination. This construction is uniform under embeddings. It uses T_r and the already established U_0; the construction of T_r used only ranks below r. Hence the recursion is well founded.

Compiling arbitrary derivations. By structural recursion on an unrestricted proof, produce a persistent procedure for its concluding sequent. Refl and Step use the procedures for their immediate subproofs. At an unrestricted Use, its antecedent procedure is already available by recursion on the proper subproof; invoke U_r at the supplied numerical target. At DE or DI, choose fresh parameters in the requested larger scope, extend the context embedding by the new clause, and invoke the previously constructed procedure for the proper branch subproof. Injective parameter renaming preserves the formula root and the literal-zero witness test, so it preserves the introduction mode.

Scope freshness and the clause-index mapping are explicit in this procedure.

This compilation recurses on original proof height. Its auxiliary T/U procedures recurse on formula rank as described above. There is no recursion on an already generated proof of uncontrolled size. Every finite input proof therefore produces a finite restricted proof. Evaluate the persistent output at the identity context embedding to obtain the required derivation. □

Implementation significance. The matched complete checker replaces `[u, index, target, premise]` by `[u, index, premise]`, removes target-term validation from Use, and reverses the context-storage order. The selected target is computed from the index and current scope. For the incoming tagged uniform-Step implementation, the complete normalized source changes from 2950 to 2938 bytes. Significant tokens remain 829, AST nodes change from 902 to 903, and informational gzip-9 size changes from 1005 to 999 bytes. This is a mathematical removal of the arbitrary specialization argument, with a small byte reduction; it is not a strict improvement in every size coordinate or an absolute minimality result.

2.4 Ordinary realization and the interpretation composites

Use the unchanged ordinary readings

$$(D_{x,y}B)^\circ = \exists x(B^\circ(x,0) \to \forall y\, B^\circ(x,y)), \qquad [A; y.B]^\circ = A^\circ \to \forall y\, B^\circ.$$

Equality is actual numerical equality. The underlying ordinary theory is intuitionistic first-order logic with equality, the scalar iterator equations, numerical coverage $\forall x(x = 0 \vee \exists y\, x = Sy)$, separation $0 = S0 \to \bot$, and induction for literal equations with side assumptions. The inherited uniform-Step equivalence identifies this ordinary theory with the unrestricted five-rule clause presentation. Its logical macro theorem recovers all intuitionistic connectives and both quantifiers; its scalar arithmetic bridge derives predecessor, reflection of equality, complete quantifier-free induction, pairing, and a compiler for every primitive recursive definition.

The new equivalence changes no term, formula, clause reading, or arithmetical axiom. It supplies proof translations in both directions on exactly the same syntax: one direction includes Use_* as the indicated ordinary Use, and the other is the finite construction in the theorem. Consequently it composes with the inherited whole-domain bi-interpretation with full first-order iPRA. The numerical domains in both directions are the whole domain, equality remains actual equality, and the numerical composite isomorphisms have graph $x = y$. The formula composites are the inherited internally provable equivalences of the logical macros; the present change adds an identity translation on formulas. Thus bi-interpretability and equiconsistency follow without a new arithmetic bootstrap. This paragraph relies on the inherited bridge theorem; the new proof above establishes the specialization equivalence itself.

Scope of the novelty claim. The new item is the restriction of clause specialization to a target computed solely from clause position and current scope, together with the persistent-procedure argument that restores arbitrary specialization and structural scope extension. The iterator, five-rule architecture, clausal contexts, scalar bridge, and arithmetic equality bootstrap are inherited. The reviewed source snapshot and the summaries of its filed packages, together with the incoming reduced-clause, root-free-clause, and uniform-Step packages, did not contain this deterministic target rule. This is a scoped comparison, not a claim of priority over all possible axiomatizations or a proof of global minimality.

3 Closing the bridge to the inherited axiomatization

The main theorem is a proof transformation between two presentations with identical formulas. This section records the remaining links to the inherited Choice-DI calculus, so that the new restriction does not rest on an unexamined claim that five rules happen to be sufficient. All derivations in this section first take place in the unrestricted clause calculus; the main compiler then transports them into the new one.

3.1 Normalization and the arithmetic interface

Zero normalization terminates: contracting $I(0, a; u.h)$ deletes its outer iterator and unused body. The recursively defined normalizer first normalizes the children and then either returns the initializer or rebuilds the iterator. This determines a unique normal form without performing any nonzero iteration. Substitution is followed by that same normalizer. Structural induction on a raw term proves that it equals its normal form using the raw zero equation and congruence. Structural induction on terms also proves coherence of normalizing substitution.

The Pivot derivation in the main theorem recovers all ordinary equality reasoning from Refl and Step. Successor computation is Step on the reflexivities of $I(c, a; u.h)$ and $I(Sc, a; u.h)$. Conversely, the raw computation equation and equality transport prove Step: replace $I(c, a; u.h)$ by t in its update, and replace the successor iterator by r. Thus no arithmetic strength is added by the uniform evaluation rule. No general primitive-recursive evaluation algorithm is part of normalization.

3.2 All case introductions and all equation inductions

The unified DI uses progress at an equation with witness literally zero. The omitted case instance is nevertheless derivable. Let

$$c = I(1, 0; u.u).$$

It is a nonliteral zero-normal term, and Step proves $c = 0$. Given a case premise $\Gamma, E(0, 0) \vdash E(0, Sz)$, begin instead with $E(c, 0)$, transport to its zero-witness instance, insert the given premise, and transport its result back to $E(c, Sz)$. Introduction at c is case mode and concludes the original $D_{x,y}E(x, y)$. Formula roots and the witness are explicit finite syntax. Ordinary-assumption cut here is proved by induction on the unrestricted derivation: a use of the distinguished constant clause is replaced by its supplied proof.

Progress at any fixed witness t is recovered by introducing $D_{w,y}E(t, y)$ with unused w and witness zero. Elimination supplies $[E(t, 0); y.E(t, y)]$. Under the desired original base, specialize it at Sz, then apply the derived case introduction at t. In particular, from a base $E(0)$ and a step under $E(z)$, derive $E(t)$ for every numerical target t. Compound progress is never introduced; that would change the induction strength.

Transport through a D formula follows by induction on its nesting depth. Eliminate the old D, introduce the new one at the fresh witness, transport its base backwards, specialize the old clause, and transport its successor instance forwards. Every recursive transport acts on the strictly smaller matrix. This gives the full formula-transport rule of the inherited Choice-DI presentation. The reverse simulation uses its arithmetic equations and transport to recover Refl and Step, and its case or progress DI to recover our selected DI. The zero-normalized and raw presentations therefore have the same translated sequents.

3.3 Why this is full intuitionistic first-order arithmetic

The primitive binder has ordinary reading

$$D_{x,y}B = \exists x(B(x,0) \to \forall y B(x,y)).$$

Although the primitive formula grammar is small, it represents all the intuitionistic connectives, including positive disjunction and existential quantification. This is stronger than showing that an implicational or negative fragment is mutually interpretable with arithmetic.

For reference, let $G_v A$ be the equation-wise selector gate and $M_v(A,B)$ the formula selector constructed in Appendix B.4. Their internally proved interfaces are

$$\vdash G_0 A, \quad G_{Sv}A \dashv\vdash A, \quad M_0(A,B) \dashv\vdash A, \quad M_{Sv}(A,B) \dashv\vdash B.$$

With $p(z) = I(z,0;u.C(u,1,0))$, the macros are

$$\begin{aligned} A \Rightarrow_* B &= D_{u,z}M_z(A,B), \\ A \wedge_* B &= D_{u,z}G_z M_{p(z)}(A,B), \\ \exists_* x\, A(x) &= D_{x,y}G_y A(x), \\ A \vee_* B &= \exists_* x\, M_x(A,B), \\ \forall_* x\, A(x) &= A(0) \wedge_* D_{u,z}G_z A(z), \\ \bot_* &= (0=1). \end{aligned}$$

Unused and freshly introduced variables are chosen to avoid capture. The appendix defines the gate and selector recursively even when the operands have different D depths, and proves their inference rules and both formula round trips. These macros belong to the external translation, not to the trusted checker.

The ordinary realization uses actual equality, the two iterator equations, numerical coverage, literal-equation induction with side assumptions, and the explicit separation bridge $(0 = S0) \to \bot$. Stating this bridge matters: positive equations alone admit a singleton structure. In the native language, contradiction is the designated equation $0 = S0$, and its explosion rule is derived; the separation bridge translates back to an identity implication.

The inherited internal arithmetic proof has a strict dependency order: scalar predecessor; actual-equality reflection; characteristic equations for all quantifier-free formulas and their induction; pairing and projections; and the translation of arbitrary primitive recursion. For a recursion with base g and step h, the final compiler stores $(n, f(n))$ in a single scalar iterator state. Its projection equations are proved internally for arbitrary elements, not checked only on standard numerals. The full proofs appear in Appendices D, E, and F.

In the opposite direction, each fixed iterator is a primitive-recursive definition in iPRA. Internal equation induction proves uniqueness of both iterator definitions and ordinary primitive recursions. Induction on a finite function description consequently proves that its double translation is the original operation. Every number remains in the domain, equality remains equality, and the identity graph is the internally provable numerical isomorphism. Formula composites use the logical macro equivalences. This establishes bi-interpretability, not merely agreement in the standard natural numbers or equiconsistency.

4 The complete checker and certificate contract

4.1 Concrete syntax and scopes

The primary checker is `code/checker.py`. It is one standalone module, with no custom imports or callbacks to the proof generator. The input is exactly a decoded JSON object with keys `goal` and `proof`. The goal must be a closed native formula. Success prints `ACCEPT` and exits with status zero; every caught invalid certificate exits with a nonzero status. The implementation does not use Python assertions for acceptance.

Object	JSON representation
Zero	`0`
Successor	`['s', t]`
Nonzero-counter iterator	`['i', n, a, h]`
Equation	`['=', s, t]`
Paired formula	`['d', B]`
Bound numerical variable	Positive integer index
Free eigenparameter	Negative integer level

Single quotes in this notation are explanatory Python notation; actual JSON files use double quotes. Zero-counter iterator nodes are rejected in supplied syntax. The opening function contracts them if substitution creates them. An iterator adds one binder only in its body. A D matrix adds two: index 1 is its y coordinate and index 2 its x coordinate. Positive indices beneath additional binders are adjusted accordingly.

At scope k the permitted free levels are $-k, \ldots, -1$. The newest is $-k$, and the next is $-k-1$. Every supplied term operand is locally closed with respect to positive indices; its free variables are the negative levels. This invariant is why opening can insert the supplied value beneath a binder without an additional shift of that value. Opening still tracks the binder depth, removes the selected index, and decrements more distant bound indices. Tests include values occurring under nested iterator and D binders.

The proof shapes are:

Schema	Certificate
Refl	`['r', t]`
Step	`['x', major, minor]`
Use_*	`['u', index, antecedent_proof]`
DE	`['e', goal, major, branch]`
DI	`['i', goal, witness, branch]`

All array arities, tags, index types, term scopes, goal scopes, and rule equalities are checked. In particular, Boolean values are not accepted as integer indices. Use verifies the antecedent before opening the clause body. Step checks the exact predecessor source and the successor shape of its minor. Branch freshness is obtained by allocation, and the branch result is checked against the target formula in the older scope, preventing eigenparameter escape.

The JSON parser has its ordinary last-key behavior for duplicate textual object keys; the contract is stated for the decoded object. As with the comparison checkers, finite runtime, memory, and

recursion limits may reject sufficiently large valid finite proofs. The universal theorems concern the corresponding finite-tree proof algorithms without such resource bounds.

4.2 Eliminating redundant scope state

The closed standalone interface begins with an empty context and scope zero. Each branch of DE or DI adds exactly one clause and allocates exactly one eigenparameter. No other recursive call changes either. Thus induction on the call tree gives

$$\texttt{scope} = |\texttt{context}|.$$

The selected checker removes the separate scope argument and computes it from the context. This is independent of deterministic specialization and can also be applied to the unrestricted control. It is an invariant of the closed public entry point; no equivalence is asserted for arbitrary external calls to the older open-context helper with mismatched arguments.

4.3 The executable proof transformation

`code/translate.py` translates an entire certificate from the exact incoming `reference/unrestricted.py` and verifies the result with the primary checker. Its reverse mode inserts the selected target and reverses the clause indices, then checks the expanded proof with the unrestricted program. Both directions preserve the final formula literally; the forward-then-reverse proof tree need not be the original tree.

Clause occurrences are represented by distinct objects in the untrusted compiler, so equal-looking clauses cannot be accidentally interchanged. A mapping sends each source eigenlevel to the fresh target level chosen under the present context embedding. A persistent premise is a function that recursively recompiles the original proper subproof for the requested extension. Formula transport consumes its D minor before entering new branches. The auxiliary recursion follows the rank order in the proof; it does not recursively translate the generated large proof.

The arithmetic constructors in `code/arith.py` are also untrusted. They produce ordinary Refl/Step trees and never certify a theorem by themselves. The checker imports neither file. This separates theorem generation from the small acceptance boundary without omitting any trusted checker component from the measurement.

5 Controlled size measurements and verification

The environment is CPython 3.12.14. A true module docstring is removed, the remaining complete module is rendered by `ast.unparse`, and one final LF is appended. Tokens exclude indentation, line-ending, comment, encoding, and end-marker tokens. AST nodes count the parsed module after docstring removal. Gzip uses level 9 and timestamp zero and is reported for information, not as the optimization objective. Raw source bytes and raw-source gzip sizes are also in `evidence/metrics.json`; the primary readable source has 3597 bytes and gzip size 1346 bytes, including its explanatory docstring.

Complete checker	Bytes	Tokens	AST	Gzip-9
Unrestricted, explicit scope	2950	829	902	1004
Deterministic, explicit scope	2938	829	903	999
Unrestricted, scope from context	2926	819	895	1000
Deterministic, scope from context	2914	819	896	997

The unrestricted matched controls store contexts newest first. This changes no source-size coordinate relative to the original incoming program; it changes gzip by one byte. The exact original source is also included and measures 2950/829/902 with gzip 1005.

The deterministic rule itself removes 12 normalized bytes, leaves the token count unchanged, and adds one AST node. The scope invariant removes 24 bytes, 10 tokens, and 7 AST nodes in either presentation. The combined change therefore improves the explicit unrestricted control by 36 bytes, 10 tokens, and 6 AST nodes. The mathematical restriction should not be described as a strict improvement in every coordinate by itself.

The historical compact 516-byte profile and the report's readable checker have different public interfaces and validation boundaries. Other filed typed presentations also use different computational normalizers. We do not assert that 2914 is the repository-wide smallest program. The separate 820-byte supplement addresses a more compact unchanged native implementation on its own matched contract.

5.1 Verification evidence

The main suite contains 100 unrestricted positive certificates: 83 inherited uniform-Step examples and 17 independently constructed stress cases. Every one was translated, accepted by the deterministic checker, expanded back, and accepted with its identical goal by the original checker. Each original, forward output, and reverse output also passed the standalone command-line interface in ordinary and optimized Python: 600 complete process checks.

The independent operational audit contains 209 frozen certificates and 956 process checks. It compares explicit and implicit scope state, checks 1200 proof mutations, rejects 93 altered false conclusions, and independently evaluates 157 accepted closed equations in the natural numbers. Its 25,047 observed explicit-scope calls all satisfy the scope invariant. Dispatch and capture tests are designed to fail when the chosen clause index or binder index is changed. The independent grammar fusion suite has 22,350 public-entry shape comparisons and 248 process checks. These numbers count executions, not disjoint mathematical theorems or a formal proof of program correctness.

The compound examples exercise D ranks one, two, and three, numerical targets containing older parameters, both coordinates bound by D, parameters under iterator binders, and antecedent proofs that themselves use compound clauses. These last cases test the persistence mechanism that a simple weakening argument would miss.

Example	Source JSON bytes	Target JSON bytes
persistent_compound_rank_3	699	175640
rank3_nested_iterator	1164	101884
rank3_older_parameter	1097	94509
induction_uses_hypothesis	324	1578

Certificate expansion is a substantial cost. A small checker need not have short proofs or fast proof

checking. No polynomial bound on the translation's serialized output is claimed. The recursion proves termination on each finite source proof; the tables report actual outputs, not a general complexity upper bound.

6 An independent 820-byte native checker refinement

This supplement preserves the original native scalar Eq/Q/D language and its entire existing certificate format. Its exact 866-byte control is `Research/scalar_uniqueness/code/compact/checker.py` at the frozen commit, Git blob `00361c25eb276ab835103d18c767e80e85b6a04d`. The full proof and portable audit are in `supplement/scope_fusion/`.

The old checker traverses a public goal once to validate its grammar and again in the kernel's mandatory first scope traversal. The refinement checks grammar during that first scope traversal. A flag is enabled only at public entry and disabled before any proof packet is read.

At a list, the added condition is exactly the old arity/sort condition. Its children receive the same depth adjustment. At an integer, the added condition is the conjunction of the old parity/sort test and the old scope test. Structural induction on the integer/list tree therefore shows that the fused entry succeeds exactly when both old passes succeed; on success it returns the same tree. The unchanged lexical input check has already excluded other types, including in unused packet prefixes. With the flag disabled, every added guard short-circuits and the remaining kernel is the original one. Neither the tree nor the proof stream has been mutated by the validation pass. Both programs consequently accept the same finite certificates, apart from incidental finite resource limits.

Complete native program	Bytes	Tokens	AST	Gzip-9
Exact control	866	381	439	504
Fused grammar/scope traversal	820	360	412	493

This is a 46-byte, 21-token, and 27-node improvement on that complete contract. It changes no logical or arithmetic rule, so its unchanged mathematical meaning is inherited directly. Its highly encoded source is less readable than the primary presentation; the two programs serve different comparison purposes. Neither one is advertised as a globally minimal axiomatization.

7 Reproduction, provenance, and limits

From the extracted directory, run

```
python3 reproduce.py
python3 code/checker.py < examples/induction.json
python3 code/translate.py < examples/compound_source.json
python3 reproduce.py --pdf
```

The first command runs the complete portable checks and refreshes the source measurements. The last additionally needs a normal TeX Live installation. The generated `report.tex` is self-contained: it includes the full theorem, inherited appendices, and checker listing, and can be compiled directly with pdfLaTeX. The readable assembly fragments are retained under `paper/`.

`CLAIMS.json` distinguishes mathematical claims, executable evidence, and limitations. `evidence/source_blobs.json` records the exact Git blobs and SHA-256 hashes of the inherited bridge files;

they match the pinned repository byte for byte. `MANIFEST.json` hashes the final distributed files. The original repository's MIT No Attribution license is included. Newly written artifacts are supplied under the same permissive terms for integration.

The new metatheorem is a written proof with independent mathematical review and executable transformations. It has not been formalized in Lean or Rocq. The repository's existing Lean developments support its inherited core; their presence is not represented as verification of our new deterministic specialization theorem. No new arithmetic-strength claim, universal function evaluator, standard-number restriction, quotient domain, or omitted logical rule is used in the argument.

A Inherited mathematical sources

The next five appendices are the authenticated source fragments from `Research/certified_zero/reference/` at the frozen commit. Their historical references to "new" or "current" refer to those inherited developments. They are included to provide the full logical and arithmetical bridge, with attribution, and are not claimed as new work of the present report.

B Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

B.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use}\quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(a,0); y.B(a,y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t,\eta_\varepsilon(z)) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

B.2 Exact equivalence and the unused-witness construction

Lemma B.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem B.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x, y)$ and its transformed premise is

$$\pi : \quad \Gamma, E(t, 0), E(t, z) \vdash E(t, Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t, Sz)$ under $\Gamma, E(t, 0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t, y)$ and all retained data. Under $\Gamma, E(t, 0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t, y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t, 0); y.E(t, y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t, 0)$ supplies its antecedent. Use at Sz gives $E(t, Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x, y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x, y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t, y)$ and binds only the unused coordinate u.

Proposition B.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

B.3 Induction and numerical cases separate cleanly

Lemma B.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma B.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D *are derivable without using the progress choice.*

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

B.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A,B), \\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B), \\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$.

Put $\delta(v) = C(v,1,0)$ and $p(z) = I(z,0;u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A,B), \\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A,B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x), \\ A \vee_* B &:= \exists_* x\, M_x(A,B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z), \\ \bot_* &:= (0 = 1). \end{aligned} \qquad \text{(Logical macros)}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A,B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A,B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A,B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \qquad \text{(Coverage)}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0,s,t) = C(1,s,t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

B.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$
$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \qquad \text{(Paired composite)}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem B.2 preserves every native sequent.

B.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

C Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section B recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

C.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.

4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem C.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

D Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [6] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

D.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

D.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n)+1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

D.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

D.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \tag{12}$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \tag{13}$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{14}$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

D.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

E From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [7]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix D, the selector is $C(n,a,b) = I(n,a;u.b)$, addition is $x+y = I(y,x;u.Su)$, and truncated subtraction is $d(x,y) = I(y,x;u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

E.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0,a,b) = a, \qquad C(Sx,a,b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n,\vec{p})$:

$$\forall \vec{p}\,\left[(E(0,\vec{p}) \wedge \forall n(E(n,\vec{p}) \longrightarrow E(Sn,\vec{p}))) \longrightarrow \forall n\, E(n,\vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

E.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

E.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

E.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)\big) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

E.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{42}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{43}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{44}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

E.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

F Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [8]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

F.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \leq b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \leq n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \leq i, \qquad R_n(i)^2 \leq n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \qquad \text{(Root invariant)}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \leq n < (q(n)+1)^2. \qquad \text{(Root)}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \leq k^2 + a < (k+1)^2$ since $a \leq k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

F.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

F.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section B give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

G Complete primary checker

The following is the entire primary acceptance program after the stated normalization. The explanatory module docstring is omitted from this listing and the size comparison; it is retained in the readable source.

```
import json
import sys

def require(condition):
    if not condition:
        raise ValueError('Invalid certificate')

def iterator(n, a, h):
    return a if n == 0 else ['i', n, a, h]

def term(a, scope, depth=0):
    if type(a) is int:
        return -scope <= a <= depth
    if type(a) is not list:
        return False
    if len(a) == 2 and a[0] == 's':
        return term(a[1], scope, depth)
    return len(a) == 4 and a[0] == 'i' and (a[1] != 0) and term(a[1], scope, depth) and term(a[2], scope,
     depth) and term(a[3], scope, depth + 1)

def formula(a, scope, depth=0):
    return type(a) is list and (len(a) == 3 and a[0] == '=' and all((term(t, scope, depth) for t in a[1:]))
     or (len(a) == 2 and a[0] == 'd' and formula(a[1], scope, depth + 2)))

def opening(a, value, depth=0):
    if type(a) is int:
        return value if a == depth + 1 else a - (a > depth + 1)
    tag = a[0]
    b = [opening(t, value, depth + (2 if tag == 'd' else int(tag == 'i' and i == 2))) for i, t in enumerate(
     a[1:])]
    return iterator(*b) if tag == 'i' else [tag] + b

def derive(proof, context):
    scope = len(context)
    match proof:
        case ['u', index, premise]:
            require(type(index) is int and 0 <= index < len(context))
            antecedent, body = context[index]
            require(derive(premise, context) == antecedent)
            return opening(body, -scope if index else ['s', 0])
        case ['r', a]:
            require(term(a, scope))
            return ['=', a, a]
        case ['x', major, minor]:
            match (derive(major, context), derive(minor, context)):
                case [['=', s, t], ['=', ['i', ['s', n], a, h], r]]:
                    require(s == iterator(n, a, h))
                    return ['=', r, opening(h, t)]
        case ['e' | 'i' as tag, goal, data, branch]:
            require(formula(goal, scope))
            fresh = -scope - 1
            if tag == 'e':
                paired = derive(data, context)
                require(paired[0] == 'd')
                body = opening(paired[1], fresh, 1)
                clause = (opening(body, 0), body)
                target = goal
            else:
                require(goal[0] == 'd' and term(data, scope))
                body = opening(goal[1], data, 1)
```

```
                eta = fresh if goal[1][0] == '=' and data == 0 else 0
                clause = (['=', 0, 0], opening(body, eta))
                target = opening(body, ['s', fresh])
            require(derive(branch, [clause] + context) == target)
            return goal
    raise ValueError('Invalid proof')

def verify(data):
    require(type(data) is dict and set(data) == {'goal', 'proof'})
    require(formula(data['goal'], 0))
    require(derive(data['proof'], []) == data['goal'])
    return True
if __name__ == '__main__':
    try:
        verify(json.load(sys.stdin))
    except Exception:
        sys.exit(1)
    print('ACCEPT')
```

References

[1] Vladimir Reshetnikov, *ProveIt: Concise presentations of intuitionistic primitive recursive arithmetic*, commit `5d85023b0688e03b75f0ec3a04bb43ef0bb51705`. Frozen ConciseIPRA subtree.

[2] ProveIt incoming research package, *One successor step for equality and computation*, `docs/incoming/ipra_uniform_step.zip`, at the commit in [1]. Exact source checker and inherited test certificates included in this package.

[3] ProveIt incoming research package, *A smaller clausal presentation of iPRA*, `docs/incoming/ipra_rootfree_clausal_2026-11-11.zip`, at the commit in [1].

[4] ProveIt incoming research package, *Reduced clause calculus*, `docs/incoming/ipra_reduced_clause_calculus_2026-11-11.zip`, at the commit in [1].

[5] ProveIt research package, *Complementary introductions with one local assumption* and *Arithmetic strength and both ordinary interpretation composites*, `Research/certified_zero/reference/inherited_choice_basis.tex` and `inherited_ordinary_bridge.tex`, at the commit in [1]. Reproduced with attribution.

[6] ProveIt research package, *Internal scalar predecessor bootstrap*, `Research/certified_zero/reference/scalar_predecessor.tex`, at the commit in [1]. Reproduced with attribution.

[7] ProveIt research package, *From literal-equation induction to full intuitionistic PRA*, `Research/certified_zero/reference/equality_reflection.tex`, at the commit in [1]. Reproduced with attribution.

[8] ProveIt research package, *Pairing, primitive recursion, and internal numerical composites*, `Research/certified_zero/reference/primitive_recursion.tex`, at the commit in [1]. Reproduced with attribution.

[9] ProveIt research package, *Scalar uniqueness*, `Research/scalar_uniqueness/code/compact/checker.py`, at the commit in [1]. Exact full-validator control for the independent scope-fusion supplement.